

13. 効率マップ・抵抗マップ読込補間

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/utils_maps.py
種別	Python
区分	model helper
役割	CSV で持つ drive/regen efficiency、Rint、OCV などを読み、補間可能な配列へ変換する。
起動文脈	SolarCarModel の map backend。

このファイルを読む理由

CSV で持つ drive/regen efficiency、Rint、OCV などを読み、補間可能な配列へ変換する。

- read_eff_map、read_Rint_map、read_map、read_1d_map が入口。
- bilinear_interp が 2D 補間の核。

呼び出し関係

どこから呼ばれるか

- mpc_solarcar/model.py

次に読むと理解が進むファイル

- 特記事項なし。

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
import numpy as np
import pandas as pd
def bilinear_interp(xg, yg, Z, x, y):
    xg = np.asarray(xg); yg=np.asarray(yg); Z=np.asarray(Z)
    x = np.clip(x, xg[0], xg[-1]); y=np.clip(y, yg[0], yg[-1])
    i = np.searchsorted(xg, x)-1; i=np.clip(i,0,len(xg)-2)
    j = np.searchsorted(yg, y)-1; j=np.clip(j,0,len(yg)-2)
    x0,x1=xg[i],xg[i+1]; y0,y1=yg[j],yg[j+1]
    Z00=Z[i,j]; Z10=Z[i+1,j]; Z01=Z[i,j+1]; Z11=Z[i+1,j+1]
    wx=0 if x1==x0 else (x-x0)/(x1-x0)
    wy=0 if y1==y0 else (y-y0)/(y1-y0)
    return (1-wx)*(1-wy)*Z00 + wx*(1-wy)*Z10 + (1-wx)*wy*Z01 + wx*wy*Z11
def read_eff_map(path):
    df = pd.read_csv(path, index_col=0)
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
def read_Rint_map(path):
    df = pd.read_csv(path, index_col=0)
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
def read_map(path):
    df = pd.read_csv(path, index_col=0)
```

主要構造の読み取り

主要関数は bilinear_interp, read_eff_map, read_Rint_map, read_map, read_1d_map。

主要クラス・関数

行	種別	名前	補足
3	function	bilinear_interp	args: xg, yg, Z, x, y
13	function	read_eff_map	args: path
16	function	read_Rint_map	args: path
20	function	read_map	args: path
24	function	read_1d_map	args: path

ファイルを上から読んだときの定義順

行	内容
3	関数 bilinear_interp を定義する。
13	関数 read_eff_map を定義する。
16	関数 read_Rint_map を定義する。
20	関数 read_map を定義する。
24	関数 read_1d_map を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import numpy as np</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L4, L5, L6, L7。
2	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L14, L17, L21, L25, L30。

関数・クラスを上から順に解説

L3 関数 `bilinear_interp`

- 定義: `bilinear_interp(xg, yg, Z, x, y)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `clip`, `len`, `searchsorted`
- 戻り値の要点: $(1 - wx) * (1 - wy) * Z00 + wx * (1 - wy) * Z10 + (1 - wx) * wy * Z01 + wx * wy * Z11$

1. `xg` に `np.asarray(xg)` の結果を代入する。
2. `yg` に `np.asarray(yg)` の結果を代入する。
3. `Z` に `np.asarray(Z)` の結果を代入する。
4. `x` に `np.clip(x, xg[0], xg[-1])` の結果を代入する。
5. `y` に `np.clip(y, yg[0], yg[-1])` の結果を代入する。
6. `i` に `np.searchsorted(xg, x) - 1` の結果を代入する。
7. `i` に `np.clip(i, 0, len(xg) - 2)` の結果を代入する。
8. `j` に `np.searchsorted(yg, y) - 1` の結果を代入する。
9. `j` に `np.clip(j, 0, len(yg) - 2)` の結果を代入する。
10. `(x0, x1)` に `(xg[i], xg[i + 1])` の結果を代入する。

L13 関数 `read_eff_map`

- 定義: `read_eff_map(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `read_csv`
- 戻り値の要点: `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))`

1. `df` に `pd.read_csv(path, index_col=0)` の結果を代入する。
2. `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))` を返す。

L16 関数 read_Rint_map

- 定義: `read_Rint_map(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `read_csv`
- 戻り値の要点: `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))`

1. `df` に `pd.read_csv(path, index_col=0)` の結果を代入する。
2. `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))` を返す。

L20 関数 read_map

- 定義: `read_map(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `read_csv`
- 戻り値の要点: `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))`

1. `df` に `pd.read_csv(path, index_col=0)` の結果を代入する。
2. `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))` を返す。

L24 関数 read_1d_map

- 定義: `read_1d_map(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `read_csv`
- 戻り値の要点: `(x, y) / (x, y)`

1. `df` に `pd.read_csv(path)` の結果を代入する。
2. 条件 `df.shape[1] >= 2` を判定し、真なら内部処理を行う。
3. `df` に `pd.read_csv(path, index_col=0)` の結果を代入する。
4. `x` に `df.index.values.astype(float)` の結果を代入する。
5. `y` に `df.iloc[:, 0].values.astype(float)` の結果を代入する。
6. `(x, y)` を返す。

処理の流れ

1. ソース中の主要関数を通じて処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/utils_maps.py` を **model helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。